Integração MongoDB + Redis com Python

Identificação

Trabalho prático — Etapa I: monitoramento ambiental por sensores IoT.

Instituição: Universidade Comunitária da Região de Chapecó (Unochapecó)

Curso: Sistemas de Informação — 4º período, turma AX

Disciplina: Banco de Dados e Novas Tecnologias — componente 1030941

Professora: Monica Tissiani de Toni Pereira

Integrantes: Thiago Antonio Virmes; João Paulo Mascarello; Erik Peter Diniz; Gabriel Correa Bussularo.

Data da verificação técnica: 3 de novembro de 2026.

Apresentação prevista: 5 de novembro de 2026.

1. Objetivo e contexto

O protótipo simula dispositivos IoT de monitoramento de PM2.5, PM10, CO_2, temperatura e umidade. O objetivo é manter um histórico de medições no MongoDB, consultar esse histórico e produzir estatísticas horárias por sensor e cidade, publicando os resumos mais recentes em Hashes no Redis. A aplicação apresenta os resultados pelo console, conforme o escopo da Etapa I.

Dados e coordenadas são ilustrativos. PM2.5 e PM10 são expressos em $\mu g/m^3$; CO_2 em ppm; temperatura em °C; umidade relativa em %. Os resultados não representam avaliação de qualidade do ar de localidades reais.

2. Arquitetura e ferramentas

Fluxo: gerador Python → MongoDB (QualidadeAr.Sensores e QualidadeAr.Medicoes) → pipelines de agregação executados no MongoDB → publicação por Python → Redis (Hashes recentes) → consulta pelo console.

O MongoDB é a fonte do histórico. Os resumos do Redis podem ser reconstruídos executando novamente a agregação. A atualização é acionada manualmente; nesta etapa não há interface gráfica, comunicação com equipamentos físicos nem processamento contínuo.

Ferramenta	Versão verificada	Uso
Windows 11 Pro x64	Ambiente local	Execução dos programas
Python	3.13.17	Gerador, console, integração e testes
MongoDB Community Server	8.0.34	Histórico e agregações
Redis	8.10.3	Hashes dos últimos resumos
PyMongo	4.19.0	Driver de acesso ao MongoDB
redis-py	8.1.0	Cliente Redis em Python
dnspython	2.9.1	Dependência do PyMongo
PowerShell e winget	Ferramentas do Windows	Instalação e inicialização

Ferramenta	Versão verificada	Uso
OpenAI Codex	Modelo específico não registrado	Apoio de IA à revisão e implementação
python-docx	1.2.0	Exportação do relatório para Word
ReportLab	5.1.0	Exportação do relatório para PDF
PyMuPDF	2.1	Conferência de conteúdo e prévia visual do PDF

Python foi instalado pelo winget a partir do pacote da Python Software Foundation. MongoDB foi obtido no site oficial. O ambiente Python do projeto foi isolado em .venv, com dependências registradas em requirements.txt.

Como não havia WSL ou Docker instalado, foi usada a compilação comunitária Redis para Windows, do repositório redis-windows, com runtime Cygwin. Ela não é uma distribuição oficial da Redis Ltd.; a orientação oficial para Redis em Windows utiliza WSL. Essa escolha viabilizou a demonstração local sem reinicialização para instalar um subsistema Linux. As referências de instalação estão na seção 10. Os arquivos dos bancos foram verificados com SHA256: MongoDB contra o checksum do fornecedor e Redis contra o digest publicado na versão do repositório.

MongoDB e Redis escutam em 127.0.0.1, nas portas 27017 e 6379. Os dados do MongoDB ficam em dados/mongo; o Redis usa AOF em dados/redis. Os executáveis portáteis ficam em .local. Não foram instalados serviços de inicialização automática; iniciar.ps1 inicia os processos locais quando necessário.

Divisão da apresentação

Gabriel Correa Bussularo apresenta a configuração do MongoDB, a modelagem e o cadastro dos sensores em mongodb.py. João Paulo Mascarello apresenta a geração de medições em mongodb.py e a consulta implementada em main.py. Erik Peter Diniz apresenta os pipelines e as estatísticas em agregacoes.py. Thiago Antonio Virmes apresenta a integração com Redis e a validação em main.py, além dos testes em test_integracao.py. Essa distribuição organiza a apresentação; não atribui autoria individual dos arquivos. As falas, comandos e transições estão em ROTEIRO_APRESENTACAO.md.

3. Modelagem no MongoDB

O banco chama-se QualidadeAr e contém as coleções Sensores e Medicoes.

Sensores possui _id textual, modelo, descrição, localização com cidade/UF/latitude/longitude, situação de atividade e lista de grandezas. Foram cadastrados S001 e S002 em Chapecó/SC; S003 e S004 em Xanxerê/SC; S005 e S006 em Concórdia/SC. A preparação usa substituição com upsert para permitir reexecução sem duplicar sensores.

Exemplo resumido do cadastro:

```
{"_id":"S001","modelo":"AirMonitor-200","localizacao":{"cidade":"Chapecó","uf":"SC","latitude":-
    27.1009,"longitude":-52.6157},"ativo":true,"grandezas":["PM2.5","PM10","CO2","temperatura","umidade"]}
```

Cada documento de Medicoes contém sensor_id, timestamp, pm25, pm10, co2, temperatura e umidade. O timestamp é armazenado como BSON Date, a partir de um datetime com fuso UTC. O _id é composto pelo sensor e instante, como S001:20261020T100000Z.

O índice composto é {sensor_id: 1, timestamp: -1}, chamado sensor_id_1_timestamp_-1. Ele atende ao padrão de consulta por sensor e período. Os resumos são calculados sob demanda e não são gravados como uma terceira coleção.

4. Geração e consulta

O gerador percorre uma grade de cinco minutos para cada sensor. Em 24 horas, cada sensor recebe 24 × 12 = 288 medições, totalizando 6 × 288 = 1.728 documentos. O período termina no início da hora UTC atual, para utilizar apenas horas completas.

Valores artificiais combinam uma variação diária e um gerador pseudoaleatório com semente derivada de sensor e instante. Assim, o mesmo sensor no mesmo instante produz os mesmos dados. As escritas usam UpdateOne com upsert e lotes de até mil operações. Reexecutar a mesma janela atualiza os registros existentes, sem multiplicar documentos. Executar em horas posteriores acrescenta histórico, de modo que o total acumulado pode superar 1.728.

O comando consultar aceita sensor ou cidade, UF opcional para a cidade e intervalo temporal. Os limites usam $gte para início e $lt para fim: [início, fim). Datas sem fuso são rejeitadas. As consultas apresentam quantidade total e uma lista limitada de registros, com unidades de medida.

Exemplos:

```
.\.venv\Scripts\python.exe main.py consultar --sensor S001 --ultimas-horas 1
.\.venv\Scripts\python.exe main.py consultar --cidade "Chapecó" --uf SC --ultimas-horas 1
```

5. Agregações e validade dos registros

Os pipelines são executados pelo MongoDB usando Medicoes.aggregate, conforme o modelo de processamento por etapas da documentação oficial. O Python recebe os resultados prontos, sem calcular as estatísticas do protótipo em memória.

Por sensor, o pipeline aplica $match, agrupa por sensor e hora com $group, projeta os campos e ordena os resultados. A hora é determinada com $dateTrunc em UTC; $dateAdd produz o fim exclusivo da janela. Os operadores exigem MongoDB 5.0 ou superior, condição conferida pelo comando verificar.

Por cidade, o pipeline associa medições e sensores com $lookup, usando sensor_id e _id, e aplica $unwind. Em seguida agrupa diretamente os registros por cidade, UF e hora. Isso calcula a média global das medições, inclusive quando os sensores possuem quantidades diferentes de registros.

Por exemplo, dois registros 10 e 30 de S001 e um registro 100 de S002 resultam em (10 + 30 + 100) / 3 = 46,6667. A média simples das médias dos sensores seria (20 + 100) / 2 = 60, inadequada para a média global solicitada.

Cada resumo contém qt_medicoes, as cinco médias, pm25_max, pm10_max, inicio_periodo e fim_periodo. A contagem usa $sum: 1; as médias usam $avg; os máximos usam $max; $project organiza o documento retornado.

São aceitos registros com identificador textual não vazio, timestamp do tipo data e as cinco grandezas numéricas escalares e finitas. PM2.5, PM10 e CO_2 precisam ser não negativos; umidade deve estar entre 0 e 100. Temperaturas negativas são válidas. Campos ausentes, tipos incorretos, arrays e valores NaN/infinito são excluídos da agregação. A regra descarta o registro inteiro quando alguma grandeza não é válida. Na agregação por cidade, registros sem cadastro correspondente de sensor são descartados pela associação.

6. Hashes no Redis

O prefixo do projeto é qualidadear:. As chaves são qualidadear:sensor:<ID> e qualidadear:cidade:<UF>:<cidade-normalizada>. A normalização remove acentos, transforma em minúsculas e substitui espaços por hífens. Exemplos: qualidadear:sensor:S001 e qualidadear:cidade:SC:chapeco.

Cada Hash contém os dez campos de resumo: os dois limites da janela, quantidade de medições, cinco médias e dois máximos. Datas são strings ISO 8601 UTC terminadas em Z; quantidades são inteiros em texto; estatísticas são textos

com duas casas decimais. O formato segue a estrutura de campos e valores dos Hashes Redis documentada na referência oficial.

Entre os resumos calculados na janela solicitada, apenas a última hora disponível de cada sensor/cidade é publicada. Uma transação Redis remove as chaves de entidades que deixaram de aparecer e substitui os Hashes atuais. A operação é uma sincronização completa da janela solicitada: deve-se utilizar uma janela que contenha todas as entidades desejadas. Para o cenário padrão são seis Hashes de sensores e três de cidades. Não há TTL; a atualização é feita explicitamente pelo comando agregar ou demonstrar.

7. Execução e resultados observados

A demonstração foi executada com sucesso em 3 de novembro de 2026, usando os bancos reais instalados neste computador. A janela registrada foi de 2026-11-02T16:00:00Z até 2026-11-03T16:00:00Z, com fim exclusivo. Novas execuções usam a janela correspondente à hora atual.

Verificação	Resultado observado
Sensores cadastrados	6 em 3 cidades
Medições na janela	1.728
Grade temporal	288 horários por sensor, espaçados de cinco minutos
Resumos horários por sensor	144, com 12 medições cada
Resumos horários por cidade	72, com 24 medições cada
Hashes recentes no Redis	9
Consulta de S001 na última hora	12 medições
Consulta de Chapecó na última hora	24 medições
Correspondência Redis/MongoDB	Campos conferidos pela validação
Testes de integração	5 aprovados

Exemplo observado para Chapecó/SC, na janela 15:00–16:00 UTC de 3 de novembro: 24 medições, PM2.5 médio de 11,39 µg/m³, PM10 médio de 35,73 µg/m³, CO_2 médio de 492,22 ppm, temperatura média de 18,26 °C e umidade média de 73,05%. PM2.5 máximo foi 15,93 µg/m³ e PM10 máximo foi 42,59 µg/m³.

As evidências completas estão em evidencias/demonstracao.txt e evidencias/testes.txt. O comando validar verifica a distribuição temporal por sensor, o total de registros válidos, a quantidade e o tamanho dos resumos em todas as 24 horas e cada campo do resumo da última hora no Redis, incluindo os dois limites do período.

Os testes usam um banco MongoDB e prefixo Redis temporários, removidos ao final. Foram verificados: separação de cidades por UF; geração idempotente e comparação independente das cinco médias e dois máximos de todos os 216 resumos; média por cidade com número desigual de registros; seleção da última hora e remoção de Hash obsoleto; exclusão de dados inválidos e do limite final, mantendo temperaturas negativas.

Para reproduzir a demonstração, basta abrir APRESENTAR.cmd. Para executar os testes, usar .\.venv\Scripts\python.exe test_integracao.py. As instruções de instalação estão no README.

8. Dificuldades encontradas e soluções

Dificuldade observada	Solução aplicada
Comando Python apontava para o atalho da Microsoft Store	Instalação do Python no perfil do usuário e uso direto do interpretador da .venv
MongoDB e Redis não estavam disponíveis	Download, verificação de integridade, extração local e script para iniciar os bancos
Windows não tinha WSL/Docker	Uso documentado de compilação comunitária Redis para Windows para a demonstração local
Filtro original rejeitava temperatura abaixo de zero	Separação das regras de temperatura, poluentes e umidade; teste com temperatura negativa
Tipos inválidos poderiam comprometer as estatísticas	Exigência de valores numéricos escalares finitos e datas válidas no filtro
Contagem total não comprovava frequência de cada sensor	Validação de toda a grade de cinco minutos e dos resumos das 24 horas
Sensores podem possuir números diferentes de registros	Agregação direta das medições por cidade e teste específico com quantidades desiguais
Reexecução poderia duplicar dados ou deixar cache antigo	Identificador determinístico, upsert e substituição dos Hashes recentes
Faltavam documentação e dependências reproduzíveis	README, versões fixadas, roteiro e este relatório com evidências e referências

Essas dificuldades correspondem à revisão e preparação realizadas nesta sessão. Não foram atribuídas à equipe experiências de desenvolvimento que não tenham sido relatadas.

9. Uso de inteligência artificial e limites

Foi utilizado OpenAI Codex como assistente de IA para analisar o enunciado e o código fornecido, localizar pendências, propor e aplicar correções, instalar ferramentas, escrever e executar testes e auxiliar na documentação. O modelo específico e eventuais usos anteriores de IA não foram informados e não são presumidos neste relatório. Esta declaração deve ser complementada pela equipe caso outras ferramentas tenham sido utilizadas.

A comprovação de funcionamento baseia-se na execução local e nos arquivos de evidência. A IA não substitui a responsabilidade dos integrantes por compreender o código, conferir a documentação, preencher a identificação e explicar o protótipo em aula.

O protótipo atende ao cenário didático configurado. Não inclui autenticação dos bancos locais, sensores físicos, agendamento ou implantação para acesso pela rede. A situação de atividade do cadastro é armazenada, mas não filtra o histórico; as estatísticas consideram medições válidas registradas no período. O relatório não estabelece limites sanitários para poluentes.

10. Referências consultadas

Consultas realizadas em 3 de novembro de 2026. O enunciado fornecido pelo usuário foi a referência dos requisitos; autoria e dados institucionais não foram informados.

• PYTHON SOFTWARE FOUNDATION. Python Releases for Windows. Disponível em: https://www.python.org/downloads/windows/ . Fonte de instalação do interpretador.

• MONGODB. MongoDB Community Downloads. Disponível em: https://www.mongodb.com/try/download/community-edition/releases . Fonte do pacote do servidor e checksum.

• MONGODB. Aggregation Pipeline. Disponível em: https://www.mongodb.com/docs/manual/core/aggregation-pipeline/ . Referência do processamento por etapas.

• MONGODB. $dateTrunc (expression operator). Disponível em: https://www.mongodb.com/docs/manual/reference/operator/aggregation/datetrunc/ . Referência das janelas horárias em UTC.

• MONGODB. MongoDB PyMongo Documentation. Disponível em: https://www.mongodb.com/docs/languages/python/pymongo-driver/current/ . Referência do driver Python.

• REDIS. Redis hashes. Disponível em: https://redis.io/docs/latest/develop/data-types/hashes/ . Referência das estruturas de campos e valores.

• REDIS. Install Redis on Windows. Disponível em: https://redis.io/docs/latest/operate/oss_and_stack/install/archive/install-redis/install-redis-on-windows/ . Orientação oficial para WSL.

• REDIS-WINDOWS. Redis for Windows. Disponível em: https://github.com/redis-windows/redis-windows . Versão usada: https://github.com/redis-windows/redis-windows/releases/tag/8.10.3 . Fonte da compilação comunitária.

• PYTHON SOFTWARE FOUNDATION. Python 3.13.12 release page. Disponível em: https://www.python.org/downloads/release/python-31312/ . Página consultada durante a pesquisa inicial; o interpretador efetivamente instalado foi o 3.13.17.

• MONGODB. Download MongoDB Community Server. Disponível em: https://www.mongodb.com/try/download/community . Página consultada para localizar o servidor Community.

• OPENAI. Codex. Disponível em: https://openai.com/codex/ . Ferramenta de IA utilizada nesta preparação; não usada como fonte de requisitos ou comprovação de execução.

• PYTHON-DOCX. Documentation. Disponível em: https://python-docx.readthedocs.io/en/latest/ . Referência da exportação para DOCX.

• REPORTLAB. ReportLab Docs. Disponível em: https://docs.reportlab.com/ . Referência da exportação para PDF.

• PYMUPDF. PyMuPDF Documentation. Disponível em: https://pymupdf.readthedocs.io/en/latest/ . Referência da conferência do documento PDF.

Apêndice — programas e scripts entregues

Os programas principais são mongodb.py, agregacoes.py e main.py. As exportações PDF e DOCX incluem seu conteúdo integral ao final. O pacote também entrega test_integracao.py, dependências e scripts de instalação/inicialização/apresentação. Os arquivos editáveis são a referência para execução e reprodução dos resultados.

Código-fonte integral — mongodb.py

```
"""Camada 1 - MongoDB: sensores, histórico e consultas.

Os dados e coordenadas são apenas ilustrativos.
"""
import math
import os
import random
from datetime import datetime, timedelta, timezone

from pymongo import ASCENDING, DESCENDING, MongoClient, UpdateOne

NOME_BANCO = "QualidadeAr"
MONGO_URI = os.environ.get("MONGODB_URI", "mongodb://127.0.0.1:27017/")


def conectar_mongo():
    cliente = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000,
                          tz_aware=True, tzinfo=timezone.utc)
    try:
        cliente.admin.command("ping")
    except Exception:
        cliente.close()
        raise
    return cliente


# Modelagem e geração de medições ilustrativas.
GRANDEZAS = ["PM2.5", "PM10", "CO2", "temperatura", "umidade"]

# Todos os pontos abaixo são ilustrativos; não são coordenadas de equipamentos reais.
SENSORES = [
    {"_id": "S001", "modelo": "AirMonitor-200", "descricao": "Estação simulada do centro de Chapecó",
     "localizacao": {"cidade": "Chapecó", "uf": "SC", "latitude": -27.1009, "longitude": -52.6157},
     "ativo": True, "grandezas": GRANDEZAS[:]},
    {"_id": "S002", "modelo": "AirMonitor-200", "descricao": "Estação simulada da zona norte de Chapecó",
     "localizacao": {"cidade": "Chapecó", "uf": "SC", "latitude": -27.0748, "longitude": -52.6240},
     "ativo": True, "grandezas": GRANDEZAS[:]},
    {"_id": "S003", "modelo": "AirMonitor-220", "descricao": "Estação simulada da região central de Xanxerê",
     "localizacao": {"cidade": "Xanxerê", "uf": "SC", "latitude": -26.8762, "longitude": -52.4047},
     "ativo": True, "grandezas": GRANDEZAS[:]},
    {"_id": "S004", "modelo": "AirMonitor-220", "descricao": "Estação simulada da região leste de Xanxerê",
     "localizacao": {"cidade": "Xanxerê", "uf": "SC", "latitude": -26.8830, "longitude": -52.3920},
     "ativo": True, "grandezas": GRANDEZAS[:]},
    {"_id": "S005", "modelo": "AirMonitor-300", "descricao": "Estação simulada da região central de
    Concórdia",
     "localizacao": {"cidade": "Concórdia", "uf": "SC", "latitude": -27.2344, "longitude": -52.0270},
     "ativo": True, "grandezas": GRANDEZAS[:]},
    {"_id": "S006", "modelo": "AirMonitor-300", "descricao": "Estação simulada da região sul de Concórdia",
     "localizacao": {"cidade": "Concórdia", "uf": "SC", "latitude": -27.2480, "longitude": -52.0380},
     "ativo": True, "grandezas": GRANDEZAS[:]},
]


def preparar_banco(banco):
    """Cria/atualiza os seis sensores sem duplicá-los e cria o índice exigido."""
    for sensor in SENSORES:
        banco.Sensores.replace_one({"_id": sensor["_id"]}, sensor, upsert=True)
    nome_indice = banco.Medicoes.create_index(
        [("sensor_id", ASCENDING), ("timestamp", DESCENDING)]
    )
    return nome_indice


def ultima_hora_fechada():
    """Limite final EXCLUSIVO (UTC) da última hora inteira concluída."""
    return datetime.now(timezone.utc).replace(minute=0, second=0, microsecond=0)


def intervalo_historico(horas=24):
    if horas <= 0:
        raise ValueError("O número de horas deve ser maior que zero.")
    fim = ultima_hora_fechada()
    return fim - timedelta(hours=horas), fim


def construir_medicao(sensor_id, instante):
    """Valores sintéticos coerentes; o mesmo sensor/instante dá o mesmo resultado."""
    deslocamento = int(sensor_id[1:])
    rng = random.Random(f"qualidade-ar:{sensor_id}:{instante.isoformat()}")
```

```
    # Variação diária para demonstrar estatísticas reais sobre dados simulados.
    ciclo = math.sin(2 * math.pi * (instante.hour + instante.minute / 60) / 24)
    pm25 = round(max(1.0, 13.0 + deslocamento * 2.2 + 6.5 * ciclo + rng.uniform(-3.5, 3.5)), 2)
    pm10 = round(pm25 + 12.0 + rng.uniform(4.0, 18.0), 2)
    co2 = round(420 + 19 * deslocamento + 45 * (ciclo + 1) + rng.uniform(0, 75), 2)
    temperatura = round(21 + 4.0 * ciclo + deslocamento * 0.25 + rng.uniform(-1.6, 1.6), 2)
    umidade = round(min(95.0, max(20.0, 66 - 9 * ciclo + rng.uniform(-7, 7))), 2)
    return {
        "_id": f"{sensor_id}:{instante.strftime('%Y%m%dT%H%M%SZ')}",
        "sensor_id": sensor_id,
        "timestamp": instante,  # datetime aware UTC -> BSON Date no MongoDB
        "pm25": pm25, "pm10": pm10, "co2": co2,
        "temperatura": temperatura, "umidade": umidade,
    }


def gerar_medicoes(banco, horas=24, reiniciar=False):
    """Gera 12 medições/hora/sensor; upsert evita duplicar timestamps."""
    if horas <= 0 or horas > 168:
        raise ValueError("Use de 1 a 168 horas.")
    inicio, fim = intervalo_historico(horas)
    if reiniciar:
        banco.Medicoes.delete_many({})
    operacoes = []
    instante = inicio
    while instante < fim:
        for sensor in SENSORES:
            documento = construir_medicao(sensor["_id"], instante)
            operacoes.append(UpdateOne(
                {"_id": documento["_id"]},
                {"$set": {campo: valor for campo, valor in documento.items() if campo != "_id"}},
                upsert=True,
            ))
        instante += timedelta(minutes=5)
    if operacoes:
        # Lotes evitam um comando muito grande quando forem geradas mais horas.
        for posicao in range(0, len(operacoes), 1000):
            banco.Medicoes.bulk_write(operacoes[posicao:posicao + 1000], ordered=False)
    return {"inicio": inicio, "fim": fim, "esperadas": len(operacoes),
            "total_no_banco": banco.Medicoes.count_documents({})}
```

Código-fonte integral — agregacoes.py

```
"""Pipelines do MongoDB: resumos horários por sensor e por cidade."""

CAMPOS = ("pm25", "pm10", "co2", "temperatura", "umidade")
CAMPOS_RESUMO = (
    "qt_medicoes", "pm25_media", "pm10_media", "co2_media",
    "temperatura_media", "umidade_media", "pm25_max", "pm10_max",
)


def filtro_validos(inicio, fim):
    """Conta somente medições com timestamp e todas as cinco grandezas numéricas."""
    filtro = {"timestamp": {"$type": "date", "$gte": inicio, "$lt": fim},
              "sensor_id": {"$type": "string", "$ne": ""}}
    # Temperaturas negativas são válidas. Limites finitos excluem NaN/Infinity.
    for campo in CAMPOS:
        filtro[campo] = {"$type": "number", "$gt": float("-inf"), "$lt": float("inf")}
        if campo != "temperatura":
            filtro[campo]["$gte"] = 0
    filtro["umidade"]["$lte"] = 100
    # $type em filtros também pode casar elementos de arrays; aqui exigimos escalares.
    filtro["$expr"] = {"$and": [
        *({"$isNumber": f"${campo}"} for campo in CAMPOS),
        {"$eq": [{"$type": "$timestamp"}, "date"]},
        {"$eq": [{"$type": "$sensor_id"}, "string"]},
    ]}
    return filtro


def expressao_hora():
    return {"$dateTrunc": {"date": "$timestamp", "unit": "hour", "timezone": "UTC"}}


def grupo_estatisticas():
    return {
        "qt_medicoes": {"$sum": 1},
        "pm25_media": {"$avg": "$pm25"},
        "pm10_media": {"$avg": "$pm10"},
        "co2_media": {"$avg": "$co2"},
        "temperatura_media": {"$avg": "$temperatura"},
        "umidade_media": {"$avg": "$umidade"},
        "pm25_max": {"$max": "$pm25"},
        "pm10_max": {"$max": "$pm10"},
    }


def projetar(campos_entidade):
    return {"_id": 0,
            **{campo: f"$_id.{campo}" for campo in campos_entidade},
            "inicio_periodo": "$_id.hora",
            "fim_periodo": {"$dateAdd": {"startDate": "$_id.hora", "unit": "hour", "amount": 1, "timezone":
    "UTC"}},
            **{campo: 1 for campo in CAMPOS_RESUMO}}


def pipeline_por_sensor(inicio, fim):
    return [
        {"$match": filtro_validos(inicio, fim)},
        {"$group": {"_id": {"sensor_id": "$sensor_id", "hora": expressao_hora()},
                    **grupo_estatisticas()}},
        {"$project": projetar(("sensor_id",))},
        {"$sort": {"sensor_id": 1, "inicio_periodo": 1}},
    ]


def pipeline_por_cidade(inicio, fim):
    return [
        {"$match": filtro_validos(inicio, fim)},
        {"$lookup": {"from": "Sensores", "localField": "sensor_id",
                     "foreignField": "_id", "as": "sensor"}},
        {"$unwind": "$sensor"},
        {"$group": {"_id": {"cidade": "$sensor.localizacao.cidade",
                            "uf": "$sensor.localizacao.uf", "hora": expressao_hora()},
                    **grupo_estatisticas()}},
        {"$project": projetar(("cidade", "uf"))},
        {"$sort": {"uf": 1, "cidade": 1, "inicio_periodo": 1}},
    ]


def agregar(banco, inicio, fim):
```

```
    """Efetivamente executa os dois pipelines no MongoDB (não agrega em Python)."""
    sensores = list(banco.Medicoes.aggregate(pipeline_por_sensor(inicio, fim)))
    cidades = list(banco.Medicoes.aggregate(pipeline_por_cidade(inicio, fim)))
    return sensores, cidades


def ultimos_resumos(registros, campos_identidade):
    """Retorna apenas a última hora disponível para cada sensor ou cidade."""
    recentes = {}
    for registro in registros:
        identificador = tuple(registro[campo] for campo in campos_identidade)
        if (identificador not in recentes or
                registro["inicio_periodo"] > recentes[identificador]["inicio_periodo"]):
            recentes[identificador] = registro
    return recentes
```

Código-fonte integral — main.py

```
#!/usr/bin/env python3
"""Projeto com 3 arquivos - entrada pelo terminal e resumos Redis.

Use: python3 main.py --help
"""
import argparse
import os
import sys
import unicodedata
from datetime import datetime, timedelta, timezone

import redis
from pymongo.errors import PyMongoError
from redis.exceptions import RedisError

from agregacoes import CAMPOS_RESUMO, agregar, filtro_validos, ultimos_resumos
from mongodb import (NOME_BANCO, SENSORES, conectar_mongo, gerar_medicoes,
                     intervalo_historico, preparar_banco)

REDIS_URL = os.environ.get("REDIS_URL", "redis://127.0.0.1:6379/0")
PREFIXO = "qualidadear:"


def conectar_redis():
    cliente = redis.from_url(REDIS_URL, decode_responses=True,
                             socket_connect_timeout=5, socket_timeout=10)
    try:
        cliente.ping()
    except Exception:
        cliente.close()
        raise
    return cliente


def normalizar_cidade(cidade):
    normalizado = unicodedata.normalize("NFKD", cidade.casefold())
    return "".join(caractere for caractere in normalizado
                   if not unicodedata.combining(caractere)).replace(" ", "-")


def chave_sensor(sensor_id):
    return f"{PREFIXO}sensor:{sensor_id}"


def chave_cidade(uf, cidade):
    return f"{PREFIXO}cidade:{uf.upper()}:{normalizar_cidade(cidade)}"


def iso_utc(data):
    if data.tzinfo is None:
        raise ValueError("Data sem fuso horário: esperado UTC.")
    return data.astimezone(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")


def mapa_resumo(resumo):
    mapeamento = {
        "inicio_periodo": iso_utc(resumo["inicio_periodo"]),
        "fim_periodo": iso_utc(resumo["fim_periodo"]),
        "qt_medicoes": str(int(resumo["qt_medicoes"])),
    }
    for campo in CAMPOS_RESUMO:
        if campo == "qt_medicoes":
            continue
        # Os valores ficam em string, como previsto no Redis.
        mapeamento[campo] = f"{float(resumo[campo]):.2f}"
    return mapeamento


def limpar_resumos(cliente_redis):
    """Remove SOMENTE os Hashes com prefixo do projeto, sem usar FLUSHDB."""
    chaves = list(cliente_redis.scan_iter(match=f"{PREFIXO}*"))
    if chaves:
        cliente_redis.delete(*chaves)
    return len(chaves)


def salvar_ultimos(cliente_redis, sensores, cidades):
    por_sensor = ultimos_resumos(sensores, ("sensor_id",))
    por_cidade = ultimos_resumos(cidades, ("uf", "cidade"))
```

```
    esperadas = {chave_sensor(sid) for (sid,) in por_sensor}
    esperadas.update(chave_cidade(uf, cidade) for (uf, cidade) in por_cidade)
    anteriores = set(cliente_redis.scan_iter(match=f"{PREFIXO}sensor:*"))
    anteriores.update(cliente_redis.scan_iter(match=f"{PREFIXO}cidade:*"))
    # Uma sincronização completa recalcula os resumos e substitui seus campos.
    with cliente_redis.pipeline(transaction=True) as pipeline:
        obsoletas = anteriores - esperadas
        if obsoletas:
            pipeline.delete(*obsoletas)
        for (sensor_id,), registro in por_sensor.items():
            pipeline.delete(chave_sensor(sensor_id))
            pipeline.hset(chave_sensor(sensor_id), mapping=mapa_resumo(registro))
        for (uf, cidade), registro in por_cidade.items():
            pipeline.delete(chave_cidade(uf, cidade))
            pipeline.hset(chave_cidade(uf, cidade), mapping=mapa_resumo(registro))
        pipeline.execute()
    return len(por_sensor), len(por_cidade)


def ler_resumos(cliente_redis):
    chaves = sorted(cliente_redis.scan_iter(match=f"{PREFIXO}*"))
    return [(chave, cliente_redis.hgetall(chave)) for chave in chaves]

def exibir_medicoes(registros):
    for doc in registros:
        print(f"  {doc['sensor_id']}  {iso_utc(doc['timestamp'])}  "
              f"PM2.5={doc['pm25']:>6.2f} µg/m³  PM10={doc['pm10']:>6.2f} µg/m³  "
              f"CO₂={doc['co2']:>6.2f} ppm  T={doc['temperatura']:>5.2f} °C  "
              f"UR={doc['umidade']:>5.2f}%")


def exibir_resumo(rotulo, dados):
    print(f"\n{rotulo} | {iso_utc(dados['inicio_periodo'])} → {iso_utc(dados['fim_periodo'])}")
    print(f"  Medições: {dados['qt_medicoes']}  "
          f"PM2.5 média/máx: {dados['pm25_media']:.2f}/{dados['pm25_max']:.2f} µg/m³  "
          f"PM10 média/máx: {dados['pm10_media']:.2f}/{dados['pm10_max']:.2f} µg/m³")
    print(f"  CO₂ médio: {dados['co2_media']:.2f} ppm  "
          f"Temperatura média: {dados['temperatura_media']:.2f} °C  "
          f"Umidade média: {dados['umidade_media']:.2f}%")

def conectar_banco():
    cliente = conectar_mongo()
    return cliente, cliente[NOME_BANCO]


def verificar():
    print("Verificando serviços locais (sem Docker)...")
    erros = 0
    try:
        mongo = conectar_mongo()
        versao = mongo.server_info()["version"]
        print(f"[OK] MongoDB acessível em localhost — versão {versao}")
        if int(versao.split(".")[0]) < 5:
            print("[ERRO] É necessário MongoDB 5.0+ para $dateTrunc e $dateAdd.")
            erros += 1
        mongo.close()
    except Exception as erro:
        print(f"[ERRO] MongoDB: {erro}")
        erros += 1
    try:
        cache = conectar_redis()
        versao = cache.info("server").get("redis_version", "desconhecida")
        print(f"[OK] Redis acessível em localhost — versão {versao} | ping=PONG")
        cache.close()
    except Exception as erro:
        print(f"[ERRO] Redis: {erro}")
        erros += 1
    return erros == 0


def preparar():
    cliente, banco = conectar_banco()
    try:
        indice = preparar_banco(banco)
        print(f"[OK] Banco {NOME_BANCO}: {banco.Sensores.count_documents({})} sensores cadastrados.")
        print(f"[OK] Índice: {indice} (sensor_id crescente, timestamp decrescente).")
        for sensor in SENSORES:
            print(f"  {sensor['_id']} → {sensor['localizacao']['cidade']}/SC")
    finally:
        cliente.close()

def gerar(horas, reiniciar=False):
```

```
    cliente, banco = conectar_banco()
    try:
        if banco.Sensores.count_documents({"_id": {"$in": [s["_id"] for s in SENSORES]}}) < 6:
            preparar_banco(banco)
        dados = gerar_medicoes(banco, horas=horas, reiniciar=reiniciar)
    finally:
        cliente.close()
    if reiniciar:
        cache = conectar_redis()
        try:
            removidas = limpar_resumos(cache)
        finally:
            cache.close()
        print(f"[OK] Limpeza solicitada: {removidas} chave(s) Redis do projeto removidas.")
    print(f"[OK] {dados['esperadas']} medições geradas/atualizadas na janela.")
    print(f"     {iso_utc(dados['inicio'])} → {iso_utc(dados['fim'])} (fim exclusivo)")
    print(f"     Total histórico no MongoDB: {dados['total_no_banco']} documentos.")
    print("     O mesmo sensor/horário não é duplicado ao reexecutar.")


def interpretar_data(entrada):
    try:
        data = datetime.fromisoformat(entrada.strip().replace("Z", "+00:00"))
    except ValueError as erro:
        raise ValueError(f"Data ISO 8601 inválida: {entrada!r}") from erro
    if data.tzinfo is None:
        raise ValueError("Informe um fuso horário; exemplo: 2026-10-20T10:00:00Z")
    return data.astimezone(timezone.utc)


def consultar(sensor_id, inicio_texto, fim_texto, ultimas_horas, limite, cidade=None, uf=None):
    if uf and not cidade:
        raise ValueError("Use --uf junto de --cidade.")
    if bool(inicio_texto) != bool(fim_texto):
        raise ValueError("Use --inicio e --fim juntos, ou apenas --ultimas-horas.")
    if inicio_texto:
        inicio, fim = interpretar_data(inicio_texto), interpretar_data(fim_texto)
    else:
        inicio, fim = intervalo_historico(ultimas_horas)
    if inicio >= fim:
        raise ValueError("O início deve ser anterior ao fim.")
    if limite <= 0 or limite > 500:
        raise ValueError("O limite deve ficar entre 1 e 500.")

    cliente, banco = conectar_banco()
    try:
        filtro = {"timestamp": {"$gte": inicio, "$lt": fim}}
        if cidade:
            filtro_local = {"localizacao.cidade": cidade}
            if uf:
                filtro_local["localizacao.uf"] = uf.upper()
            ids = [s["_id"] for s in banco.Sensores.find(filtro_local, {"_id": 1})]
            filtro["sensor_id"] = {"$in": ids}
        else:
            filtro["sensor_id"] = sensor_id
        total = banco.Medicoes.count_documents(filtro)
        registros = list(banco.Medicoes.find(filtro).sort("timestamp", 1).limit(limite))
    finally:
        cliente.close()
    entidade = f"Cidade {cidade}" if cidade else f"Sensor {sensor_id}"
    print(f"{entidade}: {total} medições em [{iso_utc(inicio)}, {iso_utc(fim)})")
    print(f"Exibindo {len(registros)} registro(s):")
    exibir_medicoes(registros)
    if total > limite:
        print(f"... {total - limite} registro(s) não exibidos. Aumente --limite.")


def atualizar_resumos(horas, mostrar_todas=False):
    inicio, fim = intervalo_historico(horas)
    cliente, banco = conectar_banco()
    try:
        resumos_sensor, resumos_cidade = agregar(banco, inicio, fim)
    finally:
        cliente.close()

    if not resumos_sensor or not resumos_cidade:
        raise ValueError("Nenhuma agregação encontrada; execute 'preparar' e 'gerar' primeiro.")
    cache = conectar_redis()
    try:
        ns, nc = salvar_ultimos(cache, resumos_sensor, resumos_cidade)
    finally:
        cache.close()
```

```
    print(f"[OK] MongoDB: {len(resumos_sensor)} resumos horários por sensor e "
          f"{len(resumos_cidade)} por cidade calculados.")
    print(f"[OK] Redis: {ns} Hash(es) de sensor + {nc} Hash(es) de cidade atualizados.")
    if mostrar_todas:
        registros_sensor, registros_cidade = resumos_sensor, resumos_cidade
    else:
        registros_sensor = list(ultimos_resumos(resumos_sensor, ("sensor_id",)).values())
        registros_cidade = list(ultimos_resumos(resumos_cidade, ("uf", "cidade")).values())
    for doc in sorted(registros_sensor, key=lambda r: (r["sensor_id"], r["inicio_periodo"])):
        exibir_resumo(f"SENSOR {doc['sensor_id']}", doc)
    for doc in sorted(registros_cidade, key=lambda r: (r["uf"], r["cidade"], r["inicio_periodo"])):
        exibir_resumo(f"CIDADE {doc['cidade']}/{doc['uf']}", doc)


def mostrar_redis():
    cliente = conectar_redis()
    try:
        hashes = ler_resumos(cliente)
    finally:
        cliente.close()
    if not hashes:
        print("Nenhum Hash do projeto. Execute 'agregar' primeiro.")
        return
    print(f"Redis: {len(hashes)} Hashes armazenados:")
    for chave, campos in hashes:
        print(f"\n{chave}")
        for campo, valor in sorted(campos.items()):
            print(f"  {campo:20} = {valor}")


def validar():
    """Confere volumes e os valores do Redis contra o MongoDB na última hora."""
    inicio, fim = intervalo_historico(24)
    ultima_inicio = fim - timedelta(hours=1)
    cliente, banco = conectar_banco()
    cache = None
    problemas = []
    try:
        cache = conectar_redis()
        ids = [sensor["_id"] for sensor in SENSORES]
        for sensor in SENSORES:
            if not banco.Sensores.find_one({"_id": sensor["_id"]}):
                problemas.append(f"Sensor {sensor['_id']} não está cadastrado.")
        quantidade = banco.Medicoes.count_documents({
            "sensor_id": {"$in": ids}, "timestamp": {"$gte": inicio, "$lt": fim}
        })
        if quantidade != 1728:
            problemas.append(f"Quantidade da janela de 24h: {quantidade} (esperado 1728).")
        # Compara toda a grade temporal: detecta lacunas, duplicatas e deslocamentos.
        grade = [inicio + timedelta(minutes=5 * i) for i in range(288)]
        for sensor_id in ids:
            filtro = {"sensor_id": sensor_id, "timestamp": {"$gte": inicio, "$lt": fim}}
            instantes = [d["timestamp"] for d in banco.Medicoes.find(filtro, {"timestamp":
    1}).sort("timestamp", 1)]
            if instantes != grade:
                problemas.append(f"{sensor_id}: esperado 288 registros na grade de cinco minutos em 24h.")
        validas = banco.Medicoes.count_documents({**filtro_validos(inicio, fim), "sensor_id": {"$in": ids}})
        if validas != 1728:
            problemas.append(f"Medições válidas na janela: {validas} (esperado 1728).")
        todas_sensor, todas_cidade = agregar(banco, inicio, fim)
        if len(todas_sensor) != 144 or any(r["qt_medicoes"] != 12 for r in todas_sensor):
            problemas.append("Esperados 144 resumos de sensor, cada um com 12 medições.")
        if len(todas_cidade) != 72 or any(r["qt_medicoes"] != 24 for r in todas_cidade):
            problemas.append("Esperados 72 resumos de cidade, cada um com 24 medições.")
        sensores, cidades = agregar(banco, ultima_inicio, fim)
        if len(sensores) != 6 or len(cidades) != 3:
            problemas.append(f"Resumos da última hora: {len(sensores)} sensores / "
                             f"{len(cidades)} cidades (esperado 6/3).")
        for registro in sensores + cidades:
            eh_sensor = "sensor_id" in registro
            chave = (chave_sensor(registro["sensor_id"]) if eh_sensor else
                     chave_cidade(registro["uf"], registro["cidade"]))
            atual = cache.hgetall(chave)
            if not atual:
                problemas.append(f"Redis: Hash ausente {chave}")
                continue
            esperado_qt = 12 if eh_sensor else 24
            if registro["qt_medicoes"] != esperado_qt:
                problemas.append(f"MongoDB {chave}: {registro['qt_medicoes']} != {esperado_qt} medições.")
            for campo, esperado in mapa_resumo(registro).items():
                if atual.get(campo) != esperado:
                    problemas.append(f"Redis {chave}: {campo} ausente ou diferente do MongoDB.")
```

```
        if len(list(cache.scan_iter(match=f"{PREFIXO}*"))) != 9:
            problemas.append("Esperados exatamente nove Hashes do projeto no Redis.")
        print(f"MongoDB: {quantidade} medições nas últimas 24h completas.")
        print(f"Agregações da hora {iso_utc(ultima_inicio)}: "
              f"{len(sensores)} sensores, {len(cidades)} cidades.")
    finally:
        cliente.close()
        if cache is not None:
            cache.close()
    if problemas:
        print("[FALHA] Validação encontrou problemas:")
        for problema in problemas:
            print("  -", problema)
        return False
    print("[OK] Todos os requisitos quantitativos e Hashes Redis conferem com o MongoDB!")
    print("[OK] 12 medições por sensor/hora; 24 por cidade/hora; 72 no total por hora.")
    return True


def demonstrar():
    print("\n==== 1/6 SERVIÇOS ====")
    if not verificar():
        raise ValueError("Verifique se MongoDB e Redis estão iniciados.")
    print("\n==== 2/6 SENSORES E ÍNDICE ====")
    preparar()
    print("\n==== 3/6 DADOS HISTÓRICOS ====")
    gerar(24)
    print("\n==== 4/6 CONSULTA NO MONGODB ====")
    consultar("S001", None, None, 1, 3)
    consultar(None, None, None, 1, 3, cidade="Chapecó", uf="SC")
    print("\n==== 5/6 AGREGAÇÃO E ATUALIZAÇÃO NO REDIS ====")
    atualizar_resumos(24)
    print("\n==== 6/6 VALIDAÇÃO DO PROTÓTIPO ====")
    if not validar():
        raise ValueError("A validação falhou; revise os dados acima.")
    mostrar_redis()
    print("\n[SUCESSO] Demonstração concluída!")


def montar_parser():
    parser = argparse.ArgumentParser(description="Estudo de caso: qualidade do ar (MongoDB + Redis).")
    sub = parser.add_subparsers(dest="comando", required=True)
    sub.add_parser("verificar", help="Testa conectividade e versões dos servidores.")
    sub.add_parser("preparar", help="Cadastra 6 sensores e cria índice MongoDB.")
    gerador = sub.add_parser("gerar", help="Gera histórico UTC a cada 5 minutos.")
    gerador.add_argument("--horas", type=int, default=24)
    gerador.add_argument("--reiniciar", action="store_true",
                         help="CUIDADO: apaga as medições e Hashes anteriores deste projeto.")
    consulta = sub.add_parser("consultar", help="Consulta medições por sensor e período UTC.")
    entidade = consulta.add_mutually_exclusive_group(required=True)
    entidade.add_argument("--sensor")
    entidade.add_argument("--cidade", help="Nome da cidade como cadastrado, com acentos.")
    consulta.add_argument("--uf", help="UF opcional para consulta por cidade.")
    consulta.add_argument("--inicio", help="ISO UTC, exemplo 2026-10-20T10:00:00Z")
    consulta.add_argument("--fim", help="ISO UTC, exemplo 2026-10-20T11:00:00Z")
    consulta.add_argument("--ultimas-horas", type=int, default=2)
    consulta.add_argument("--limite", type=int, default=12)
    agreg = sub.add_parser("agregar", help="Agrega horários em MongoDB e grava últimos Hashes Redis.")
    agreg.add_argument("--horas", type=int, default=24)
    agreg.add_argument("--mostrar-todas", action="store_true", help="Mostra todas as horas; saída longa.")
    sub.add_parser("redis", help="Mostra os Hashes gravados no Redis.")
    sub.add_parser("validar", help="Valida requisitos e compara Redis com MongoDB.")
    sub.add_parser("demonstrar", help="Executa uma demonstração completa para apresentar.")
    return parser


def main():
    args = montar_parser().parse_args()
    try:
        if args.comando == "verificar":
            return 0 if verificar() else 1
        if args.comando == "preparar":
            preparar()
        elif args.comando == "gerar":
            gerar(args.horas, args.reiniciar)
        elif args.comando == "consultar":
            consultar(args.sensor, args.inicio, args.fim, args.ultimas_horas, args.limite,
                      args.cidade, args.uf)
        elif args.comando == "agregar":
            atualizar_resumos(args.horas, args.mostrar_todas)
        elif args.comando == "redis":
            mostrar_redis()
```

```
        elif args.comando == "validar":
            return 0 if validar() else 1
        elif args.comando == "demonstrar":
            demonstrar()
        return 0
    except (PyMongoError, RedisError, ValueError, OSError) as erro:
        print(f"[ERRO] {erro}", file=sys.stderr)
        return 1
    except KeyboardInterrupt:
        print("\nExecução interrompida.", file=sys.stderr)
        return 130


if __name__ == "__main__":
    raise SystemExit(main())
```